

Контрольная работа: Информационная система «Медицинские напоминания»

1. Анализ предметной области

Описание предметной области: Данная система предназначена для автоматизации процесса контроля приема лекарственных препаратов пациентами с хроническими заболеваниями. Основная проблема, которую решает система — низкая приверженность лечению из-за сложности графиков приема и человеческого фактора (забывчивость).

Выполняемые программой функции:

- **Хранение и управление данными:** Ведение реестров пациентов, врачей, справочников препаратов и диагнозов.
- **Назначение лечения:** Создание врачом индивидуальных курсов лечения с указанием дозировок и сроков.
- **Планирование приема:** Формирование детального графика (время суток, дни недели) для каждого препарата.
- **Мониторинг и уведомления:** Генерация напоминаний о приеме и фиксация факта принятия препарата пользователем.

Нефункциональные требования:

- **Надежность:** Обеспечение целостности медицинских данных (ACID в PostgreSQL).
- **Производительность:** Высокая скорость доступа к текущему расписанию и управлению сессиями (использование Redis).
- **Безопасность:** Разграничение прав доступа между Администратором, Врачом и Пациентом.

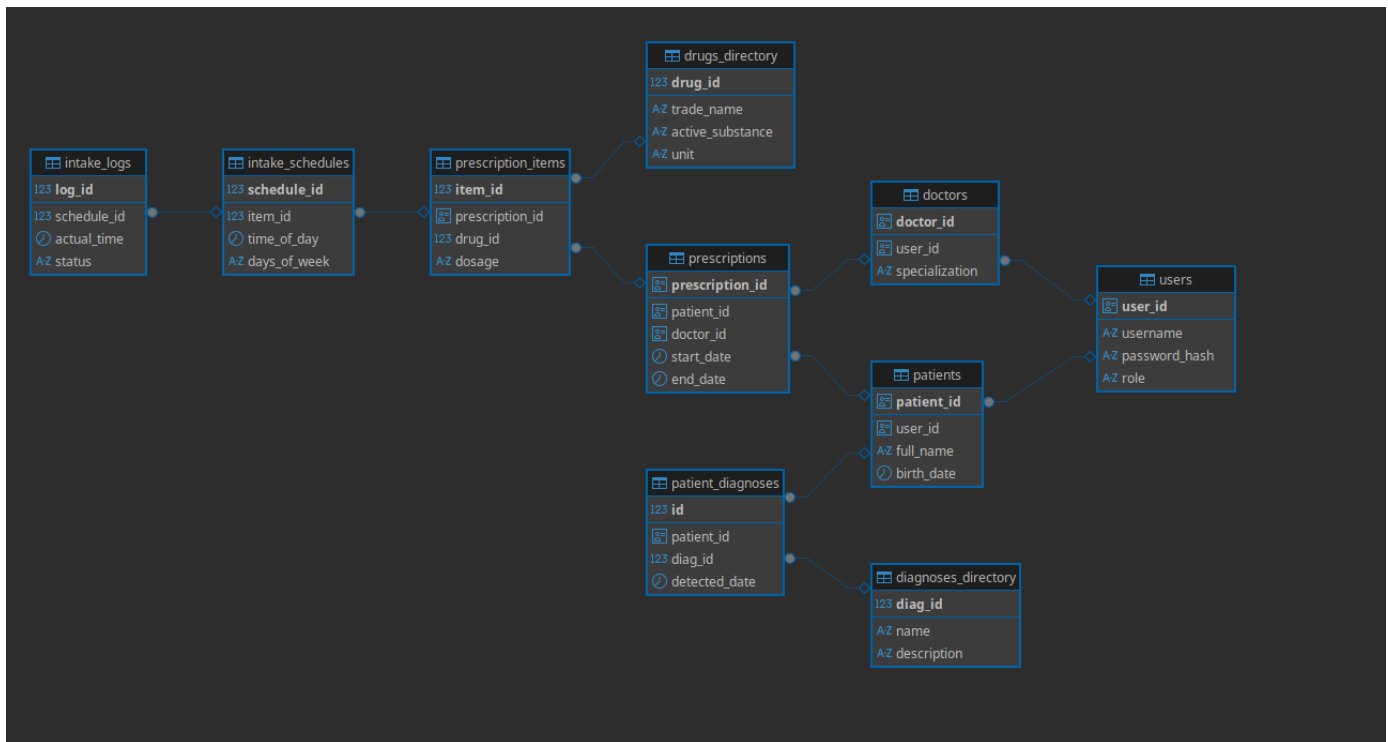
2. Концептуальная модель БД и ER-диаграмма

Система базируется на подходе **Polyglot Persistence**, используя две разные СУБД для разных типов данных.

Реляционная БД (PostgreSQL)

Используется для хранения структурированных данных с жесткими связями.

- **Сущности:** Пользователи, Пациенты, Врачи, Справочники диагнозов и лекарств, Назначения, Графики приема, Логи приемов.
- **Связи:** Один-ко-многим (Один врач → много назначений; один пациент → много назначений).



NoSQL БД (Redis)

Используется как высокопроизводительный слой для оперативных данных.

- **Типы данных:**
 - **Strings** : Для хранения токенов активных сессий пользователей.
 - **Hashes** : Для быстрого кэширования справочной информации о препаратах.
 - **Lists** : Для реализации очереди уведомлений.

3. Логическая схема БД

База данных спроектирована с соблюдением 3-й нормальной формы. Основные ограничения включают:

- **PRIMARY KEY** для каждой таблицы (UUID для обеспечения уникальности в распределенных системах).
- **FOREIGN KEY** для обеспечения ссылочной целостности (например, назначение не может существовать без пациента и врача).
- **CHECK** ограничения для ролей пользователей и статусов приемов.

(Подробная схема таблиц представлена в приложениях к отчету / Lab 1)

4. Расширенные возможности реляционной СУБД

В рамках реализации были применены следующие инструменты оптимизации PostgreSQL:

- **Индексация:** Созданы B-tree индексы для часто запрашиваемых полей (ФИО пациентов, названия препаратов), что позволило существенно сократить время выполнения поисковых запросов.

- **Программируемость (PL/pgSQL):** Реализованы хранимые функции и процедуры для автоматизации бизнес-логики, такие как расчет следующего времени приема препарата.
- **Представления (Views):** Созданы сложные представления для быстрого получения «Дневного расписания» и «Нагрузки врачей».

5. Реализация и наполнение БД

БД развернуты с помощью Docker Compose. Наполнение данными осуществлялось двумя способами:

1. **SQL-скрипты:** Первичная инициализация структуры и базовых справочников (`init_db.sql`).
2. **Python-генератор:** Автоматическое создание набора из 60+ записей для каждой таблицы для имитации реальной нагрузки и тестирования производительности.

6. Операции с базами данных

PostgreSQL

Реализован полный цикл CRUD:

- **Добавление:** Создание новых пациентов и назначений.
- **Редактирование:** Обновление дозировок и дат курса.
- **Удаление:** Отмена назначений.
- **Сложные выборки:** Использование `JOIN` для объединения данных о пациентах, препаратах и графиках.

Redis

- **Управление сессиями:** `SET session:token user_id EX 3600 → GET session:token` .
- **Кэширование:** Использование `HSET` и `HGETALL` для быстрого доступа к данным о лекарствах.
- **Очереди:** `LPOP` и `RPOP` для управления потоком уведомлений.

7. Интерфейс информационной системы

Для системы реализован консольный интерфейс (CLI), обеспечивающий доступ ко всем функциям управления.

Проектирование интерфейса (Макеты):

☰

СИСТЕМА НАПОМИНАНИЙ

👤

Профиль

12.06.2026

Привет, Иван Иванович! 🙌

Сегодня у вас 3 приема препаратов:

[]

Эналаприк (5мг)

08:00

[v]

← Карточка препарата

[]

Аспирин (100мг)

12:00

[v]

[x]

Эналаприк (5мг)

08:00

[v]

← Отмечено как "принято"

🏠

Главная

🕒

История

⚙️

Настройки

← Нижняя навигация

[← Назад] ОФОРМЛЕНИЕ НАЗНАЧЕНИЯ

ПОИСК ПАЦИЕНТА:

[Иванов Иван Иванович]

ДАННЫЕ НАЗНАЧЕНИЯ:

Выберите препарат:

[Выберите из списка...] ← (Справочник drugs_dir)

Дозировка:

[5 мг]

Период курса:

С [12.06.2026] по [12.07.2026]

График приема:

Время: [08:00]

Дни: [Пн] [Вт] [Ср] [Чт] [Пт] [Сб] [Вс] ← (Чекбоксы дней)

[СОХРАНИТЬ НАЗНАЧЕНИЕ] ← Главная кнопка

[≡] АДМИНИСТРАТОР

Управление данными

ВЫБЕРИТЕ СПРАВОЧНИК:
[Справочник Лекарств] [Справочник Диагнозов]

Поиск по названию: Метформин...

ID	Название	Вещество	Ед.изм	Действия
1	Эналаприк	Эналаприлат	таб	[Edit] [Delete]
2	Метформин	Метформин	таб	[Edit] [Delete]
3	Аспирин	Ацетилсалиц.	таб	[Edit] [Delete]

[+ ДОБАВИТЬ НОВЫЙ ПРЕПАРАТ]

Реализованный интерфейс приложения: Главное меню управления системой:

```
=====
Medical Reminders System - Admin Panel
=====
1. 👤 Patients Management
2. 💊 Prescriptions Management
3. 🔑 Session & Cache (Redis)
4. 📊 Functional Reports
0. ❌ Exit

Select an option:
```

8. Архитектура программы

Приложение построено на принципах многослойной архитектуры и ООП.

Схема архитектуры:

UI Layer (main.py) → Service Layer (medical_service.py) → Repository Layer (repositories.py)
→ Database (PostgreSQL/Redis)

Диаграмма классов (High-level):

```
classDiagram
    class MedicalService {
        +create_patient()
        +list_prescriptions()
        +get_patient_full_schedule()
        +start_session()
    }
    class PostgresRepository {
        <<abstract>>
        +conn
        +get_cursor()
    }
    class PatientRepository {
        +create()
        +get_all()
        +update()
    }
    class PrescriptionRepository {
        +get_all_with_patients()
        +get_patient_schedule()
    }
    class RedisRepository {
        +client
    }
    class SessionRepository {
        +create_session()
        +get_session()
    }

    MedicalService --> PatientRepository
    MedicalService --> PrescriptionRepository
    MedicalService --> SessionRepository
    PatientRepository --|> PostgresRepository
    PrescriptionRepository --|> PostgresRepository
    SessionRepository --|> RedisRepository
```

Алгоритм получения расписания:

1. Пользователь вводит `patient_id` .
2. `MedicalService` запрашивает данные у `PrescriptionRepository` .
3. Выполняется SQL-запрос с JOIN четырех таблиц: `patients` → `prescriptions` → `prescription_items` → `drugs_directory` → `intake_schedules` .
4. Результат возвращается в UI в виде списка препаратов с временем приема.

9. Программная реализация

- **Язык:** Python 3.11.
- **Библиотеки:** psycopg2-binary (PG), redis (NoSQL).
- **Реализация функционала:**
 - Реализован полный CRUD для основных сущностей.
 - Интегрированы сложные JOIN-запросы для вывода атрибутов вместо ID.
 - Внедрена поддержка сессий и кэширования через Redis.

Демонстрация работы (скриншоты):

- **CRUD и JOIN:**

```
=====
                Prescriptions Management
=====
1. List all prescriptions (with Patients)
2. Create prescription
3. Update prescription
4. Delete prescription
0. Back

Option: 1

--- Prescriptions ---
ID: 591763c4-237e-4966-8f2d-0f40d3cccf57 | Patient: Updated Test User | From: 2026-06-01 To: 2026-06-30
ID: 8419d158-a747-4bc5-8a01-0770dbbc4a94 | Patient: Ivan Ivanov | From: 2026-06-01 To: 2026-07-01
```

- NoSQL функции:

```
=====
                        Redis Session & Cache
=====

1. Create Session
2. Check Session
3. Delete Session
4. Cache Drug Info
5. Get Cached Drug
0. Back

Option: 1
Token: 123
User ID: ramzik
Session started.

=====
                        Redis Session & Cache
=====

1. Create Session
2. Check Session
3. Delete Session
4. Cache Drug Info
5. Get Cached Drug
0. Back

Option: 2
Token: 123
Session belongs to User: ramzik
```

- **Функциональный отчет:**

```
=====
      Medical Reminders System - Admin Panel
=====
1. 👤 Patients Management
2. 💊 Prescriptions Management
3. 🔑 Session & Cache (Redis)
4. 📊 Functional Reports
0. ❌ Exit

Select an option: 4

=====
      Functional Reports
=====
1. Patient's Daily Schedule
2. Drug Popularity Statistics
0. Back

Option: 1
Patient ID: 3b235740-3acc-4255-b7ad-80cd7e59350c

--- Schedule for Patient 3b235740-3acc-4255-b7ad-80cd7e59350c ---
💊 Enalapril | Time: 08:00:00 | Dose: 5mg
```

10. Состав проекта

Итоговый пакет содержит:

- **Исходный код:** Все файлы из директории `/src` (Python).
- **Базы данных:** Скрипты инициализации (`init_db.sql`) и наполнения (`seed_data.sql` , `seed_redis.sh`).
- **Документация:** Данный отчет.
- **Репозиторий:** <https://github.com/Ramzikkkk/medical-reminders-db.git>